

EXAMEN C++ – VERSION C

Master Sciences pour l'Ingénieur

Durée : 1 heure

Consignes :

- Répondez aux **2 exercices** ci-dessous
- La durée totale est d'**une heure**
- Vous pouvez utiliser du papier brouillon pour le pseudo-code
- **Compilabilité obligatoire** : votre code doit compiler sans erreur
- Lisibilité et commentaires sont appréciés

Important

- Préférez `const T&` dans les signatures quand approprié
 - Assurez-vous que vos destructeurs gèrent bien la mémoire
-

Exercice 1 : Hiérarchie de Véhicules [10 points]

Contexte

Vous devez modéliser une flotte de véhicules. Chaque véhicule a une consommation de carburant différente selon son type.

Questions

1. [3 points] Créez une classe abstraite `Vehicle` avec :
 - Des membres protégés : `string brand` et `double tankCapacity` (en litres)
 - Une méthode virtuelle pure `double fuelConsumption()` retournant la consommation en L/100km
 - Une méthode virtuelle `void displayInfo()` affichant la marque et la capacité
 - Un destructeur virtuel
2. [3 points] Créez deux classes concrètes :
 - `Car(string brand, double tankCapacity, double consumption)` avec :
 - `fuelConsumption()` retourne `consumption`
 - `displayInfo()` affiche la marque, la capacité et le type "Voiture"
 - `Truck(string brand, double tankCapacity, double loadTons)` avec :
 - `fuelConsumption()` retourne $20.0 + 2.5 \times \text{loadTons}$
 - `displayInfo()` affiche la marque, la capacité, le type "Camion" et le tonnage
3. [2 points] Surchargez l'opérateur `>` pour comparer deux véhicules.
`v1 > v2` retourne `true` si `v1` consomme **plus** que `v2`.
4. [2 points] Écrivez un `main()` qui :
 - Crée une `Car` : "Toyota", réservoir 50L, consommation 6.5 L/100km

- Crée un `Truck` : "Renault", réservoir 300L, charge 5.0 tonnes
- Affiche les infos et la consommation de chaque véhicule
- Affiche lequel consomme le plus via l'opérateur `>`

Exercice 2 : Template Convertisseur d'Unités [10 points]

Contexte

Vous devez créer un template `Converter<T>` capable de convertir des valeurs numériques d'une unité à une autre (km en miles, Celsius en Fahrenheit, kg en livres...) à l'aide d'un facteur et d'un offset.

Questions

1. [4 points] Créez un template de classe `Converter<T>` avec :
 - Deux membres privés : `T factor` et `T offset`
 - Un constructeur `Converter(T factor, T offset = 0)`
 - Une méthode `T convert(T value)` appliquant :

$$\text{résultat} = \text{value} \times \text{factor} + \text{offset}$$

- Des getters `T getFactor()` et `T getOffset()`
2. [3 points] Surchargez l'opérateur `*` pour enchaîner deux convertisseurs.
`c1 * c2` retourne un nouveau `Converter<T>` tel que :
 - `factor = f1 × f2`
 - `offset = offset1 × f2 + offset2`*Cela revient à appliquer c1 puis c2 en une seule conversion.*

3. [3 points] Écrivez un `main()` qui :
 - Crée un `Converter<double>` km → miles (factor = 0.621371, offset = 0)
 - Crée un `Converter<double>` Celsius → Fahrenheit (factor = 1.8, offset = 32.0)
 - Convertit 100 km en miles et affiche le résultat
 - Convertit 0°C et 100°C en Fahrenheit et affiche les résultats
 - Enchaîne les deux convertisseurs avec `*` et affiche le `factor` et l'`offset` du résultat